SMART ELECTRICITY MANAGEMENT Using IOT And Machine Learning

1. Introduction:

"Smart Electricity Management" is an innovative system designed to optimize energy consumption by leveraging machine learning techniques.

This system integrates human detection, temperature monitoring, and light intensity measurement to intelligently control electrical devices, ensuring energy efficiency and sustainability.

By incorporating human detection, the system identifies the presence or absence of individuals in a space, allowing for dynamic control of electrical appliances. For example, lights and air conditioning systems can be activated only when humans are detected, significantly reducing unnecessary power usage.

Temperature monitoring is another critical component of the system. It ensures that heating, ventilation, and air conditioning (HVAC) systems operate based on real-time environmental conditions, maintaining comfort while minimizing energy waste.

The measurement of light intensity enables the system to adjust artificial lighting according to natural light availability. This not only enhances energy efficiency but also creates a comfortable and well-lit environment.

Machine learning techniques empower the system to learn and adapt over time, predicting usage patterns and optimizing energy management decisions. By combining these advanced technologies, **Smart Electricity Management** offers a sustainable and intelligent solution to reducing energy consumption, lowering costs, and contributing to environmental conservation.

2. Objective:

The primary objective of **"Smart Electricity Management"** is to optimize energy usage and enhance efficiency in electricity consumption through advanced technology and intelligent systems. This can be achieved by monitoring, controlling, and automating electrical systems to reduce waste, lower costs, and promote sustainable energy practices. The below are the specific goals:

- **Energy Efficiency**: Minimize energy wastage by optimizing power usage in homes, industries, any organization, commercial spaces and wherever the electricity is used by humans.
- **Cost Reduction**: Reduce electricity bills by identifying and curbing unnecessary consumption and using energy-saving techniques.
- **Automation**: Implement smart devices and systems that can autonomously monitor and control electricity usage based on real-time conditions or predefined settings.
- **Sustainability**: Support the use of renewable energy sources and integrate them into the energy management system to promote eco-friendly practices.
- **Real-time Monitoring**: Enable users to track electricity consumption in real-time using IoT devices, sensors, and data analytics.

These objectives collectively aim to create smarter, greener, and more efficient electricity management solutions.

3.Hardware and Software Support:

- **Hardware**:

- **Microcontroller**: Arduino UNO or Raspberry Pi used for data processing and controlling the Peripheral devices.
- **DHT11 Temperature and Humidity Sensor**: It can monitor the temperature and humidity in real time.
- **LDR Sensor**: It read light intensity in real time.
- **Jumper Wires**: These wires are used as circuit connection between peripheral devices.
- **DC Motor** : Instead of using external fan we use the DC motor as fan for practical appliance.
- **LED** : LED is used for light systems.
- **Camera** : Camera is used for real time image capturing based on image data peripheral devices are controlled.
- **Power Supply**: A reliable power source for system operation.

➢ **Software:**

❖ **Programming Language:** Python

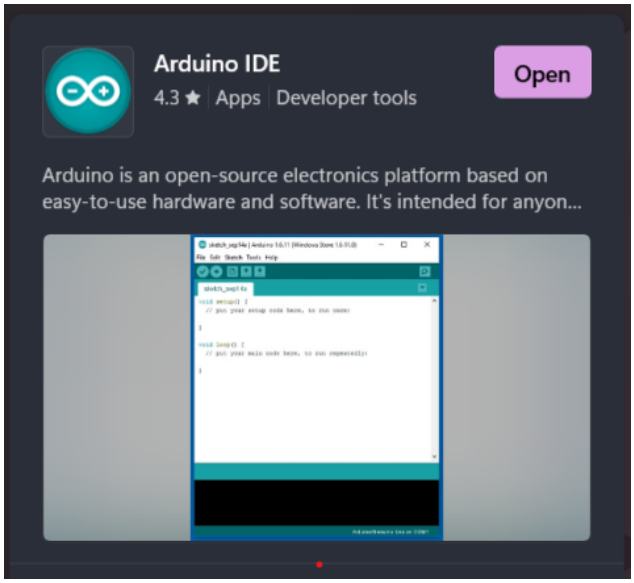
❖ **Framework and Libraries:**

- **PyTorch:** For deep learning tasks, particularly object detection using MS COCO dataset.
- **YOLOv5-master**: A state-of-the-art object detection model used for identifying Humans.

❖ **GPU Acceleration**: CUDA, for high-speed processing and real-time performance.

❖ Development Tools:

- **Arduino IDE**: Arduino Ide for microcontroller programming and uploading the code.



- Python or similar programming languages for advanced data processing.

4. Working Principle:

The working principle of **Smart Electricity Management** revolves around the seamless integration of human detection, temperature monitoring, and light intensity measurement, all powered by machine learning techniques. Here's how it functions step-by-step:

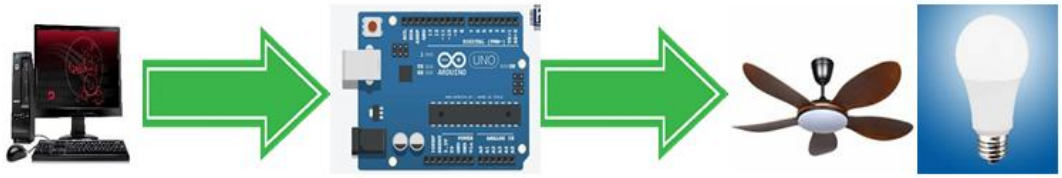
- **Real-Time Data Collection:**
  - Temperature sensor and LDR sensor sense the light intensity and temperature in the installed environment.
  - Cameras capture video footage and send it to the Computer or Raspberry Pi for analysis.

Person Image CCTV Camera Computer

- **Data Analysis and Processing:**
  - YOLOv5 processes the video feeds and identifies objects like Human.
  - The Python-based control system calculates person count in the environment.

Before Detection Image Image Processing After Detection Image

- **Signal Optimization:**
  - Based on the collected data, the system dynamically adjusts light and ventilation system in the environment.



- **Human Detection**:
  The system uses advanced sensors (e.g., cameras with human detection models such as YOLO) to identify the presence or absence of individuals in a specific area.
  - **If a human is detected**, the system activates or maintains the operation of electrical appliances (e.g., lights, fans, air conditioning).
  - **If no human is detected**, it switches off or reduces the functionality of appliances to conserve energy.

IMPLEMENTATION OF YOLOv5 MODULE

To set up and implement the YOLOv5 object detection module, follow these step-by-step instructions:

Step 1: Install YOLOv5 from GitHub

1. **Download the YOLOv5 Repository:**
   - Visit the official YOLOv5 GitHub page at https://github.com/ultralytics/yolov5.
   - Click the **Code** button and select **Download ZIP** to download the repository.

- Once downloaded, extract (unzip) the ZIP file to a location on your system.

2. **Prepare the YOLOv5 Module:**
   - Ensure you have a terminal or command prompt with Python and Git installed.
   - Navigate to the YOLOv5 folder using the terminal:

```
cd path/to/yolov5
```

   - Command : cd path/to/yolov5

Step 2: Install Python 3.8

1. **Download Python 3.8:**
   - Go to the official Python website: https://www.python.org.
   - Download Python 3.8, as it is highly compatible with YOLOv5 and the MS COCO dataset.
   - Install Python 3.8 and ensure that the **Add Python to PATH** option is checked during installation.
2. **Verify Installation:**
   - Open the terminal and type:
   - Command: python –version

```
python --version
```

   - Ensure it displays Python 3.8.x.

Step 3: Install PyTorch

1. **For CPU Users:**

- Install PyTorch by entering the following command in the terminal:
- Command: pip install torch torchvision torchaudio

```
pip install torch torchvision torchaudio
```

2. **For GPU Users:**
   - Ensure CUDA is installed. CUDA is essential for leveraging GPU acceleration for PyTorch.
   - To install CUDA, visit NVIDIA CUDA Toolkit and download the version compatible with your GPU.
   - After installing CUDA, install PyTorch with GPU support by entering:
   - Command: conda install pytorch torchvision torchaudio pytorch-cuda=11.8 -c pytorch -c nvidia

```
conda install pytorch torchvision torchaudio pytorch-cuda=11.8 -c pytorch -c nvidia
```

3. **Verify PyTorch Installation:**
   - Open a Python interpreter and type:

```
import torch
print(torch.cuda.is_available()) # Should return
True if GPU is properly configured
```

Command: import torch

- print(torch.__version__)
- print(torch.cuda.is_available()) # Should return True if GPU is properly configured

Step 4: Install MS COCO API

1. **Install the COCO API:**
    - Run the following command to install the COCO API, which is required for working with the MS COCO dataset:

```
pip install git+https://github.com/philferriere/cocoapi.git#subdirectory=PythonAPI
```

    - Command: pip install git+https://github.com/philferriere/cocoapi.git#subdirectory=PythonAPI

2. **Verify Installation:**
    - Test the installation by importing the COCO API in Python:

```
from pycocotools.coco import COCO
```

    - Command: from pycocotools.coco import COCO

Step 5: Install YOLOv5 Requirements

1. **Navigate to the YOLOv5 Directory:**
    - Use the terminal to navigate to the extracted YOLOv5 folder:

```
cd path/to/yolov5
```

    - Command: cd path/to/yolov5

2. **Install Required Libraries:**
    - Run the following command to install all dependencies listed in the requirements.txt file:

```
pip install -r requirements.txt
```

- Command: pip install -r requirements.txt

3. **Verify YOLOv5 Installation:**
   - Test YOLOv5 by running a quick inference

```
python detect.py --source 0 # Use 0 for webcam, or replace with an image/video file path
```

   - Command: python detect.py --source 0 # Use 0 for webcam, or replace with an image/video file path.
4. Ensure your environment has an internet connection for downloading dependencies.
5. If you encounter errors during setup, carefully review the terminal output and install any missing packages manually using pip.
6. For GPU acceleration, verify that your GPU drivers and CUDA installation are up-to-date.

This detailed setup ensures your YOLOv5 module is ready for object detection tasks using MS COCO or custom datasets.

Hardware Connections:

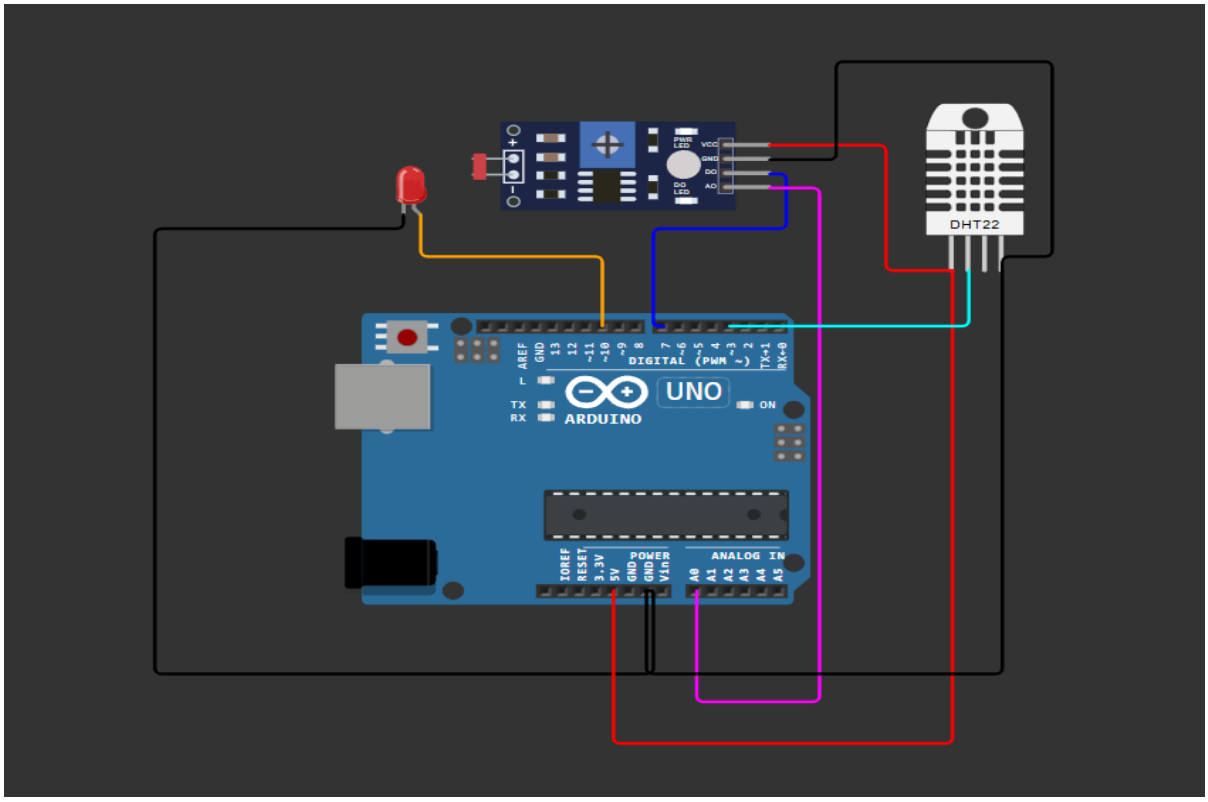
Here's a structured step-by-step implementation plan for this project:

1. **LED and Fan connection(DC Motor) to Arduino UNO**:
   - **GND** (LED) → **GND** (Arduino)

- **GND (**DC Motor**)** → **GND**(Arduino)
- **Positive** (LED) → **GPIO Pin 10** (Arduino): Detect the presence of Human.
- **VCC(LDR)** → **5V**(Arduino): Power supply to LDR
- **VCC(DHT11)** → **5V**(Arduino): Power supply to DHT11 Temperature and Humidity sensor
- **GND(LDR)** → **GND**(Arduino): Power supply to LDR
- **GND(DHT11)** →**GND**(Arduino): Power supply to DHT11.
- **Data(LDR)** → **GPIO Pin 2** (Arduino): Detect the Light intensity.
- **Data(DHT11)** → **GPIO Pin 7** (Arduino): Detect the Temperature and Humidity.
- **Positive** (LED) →**GPIO Pin 11** (Arduino):For light appliances.
- **Positive**(DC Motor) → **GPIO Pin 4** & **GPIO Pin 5** & **GPIO Pin 6**(Arduino):For ventilation like fan.

2. **USB Connection**:

- Use a **USB A/B** or **Mini B cable** to connect your Arduino UNO to the computer for code uploading and serial communication.



Code Setup

1. Arduino Code

- Open the Arduino IDE and upload the Demo_LED_Test6.ino code to your **Arduino UNO**

```
#define LED_PIN 10

// Pin connected to the LED light

#include "DHT.h"


#define DHTPIN 2 //it reading input data from temperature sensor

#define DHTTYPE DHT11   // DHT 11 for temperature

int fan1=5;

int fan2=6;

int fan3=4;

// It operates fan ON and OFF
```

```
int ldr =7; // It operatus the light pin
int x;
int led=11;
DHT dht(DHTPIN, DHTTYPE);



void setup() {
  pinMode(LED_PIN, OUTPUT); //out pin
  pinMode(ldr,INPUT);//input pin of LDR
  pinMode(led,OUTPUT);//Output pin of Led
  pinMode(fan1,OUTPUT);//Output pin of Fan1
  pinMode(fan2,OUTPUT);//Output pin of Fan2
  pinMode(fan3,OUTPUT);//Output pin of Fan3
  Serial.begin(9600);
  Serial.println(F("DHTxx test!"));
  dht.begin();
}

void loop() {

  // temperature measuring


  //delay(2000);


  // Reading temperature or humidity takes about 250 milliseconds!
  // Sensor readings may also be up to 2 seconds 'old' (its a very slow sensor)
  float h = dht.readHumidity();//reads the humidity
  // Read temperature as Celsius (the default)
  float t = dht.readTemperature();
  // Read temperature as Fahrenheit (isFahrenheit = true)
```

```
float f = dht.readTemperature(true);


// Check if any reads failed and exit early (to try again).
if (isnan(h) || isnan(t) || isnan(f)) {
  Serial.println(F("Failed to read from DHT sensor!"));
  return;
}
// Compute heat index in Fahrenheit (the default)
float hif = dht.computeHeatIndex(f, h);
// Compute heat index in Celsius (isFahreheit = false)
float hic = dht.computeHeatIndex(t, h, false);
Serial.print(F("Humidity: "));
Serial.print(h);
Serial.print(F("%  Temperature: "));
Serial.print(t);
Serial.println();
//
x=digitalRead(ldr);//It storethe  it in the variable 'x'
Serial.println(x);
if (Serial.available() > 0)//It recieves the data from serial port
{
  char signal = Serial.read();


  if (signal == '1') //human is found
  {
     if(x==HIGH)//checks the light intensity and turn ON the led


            digitalWrite(led,HIGH);
        }
     if(x==LOW)//checks the light intensity and turn OFF the led
```

```
        {
          digitalWrite(led,HIGH);
        }
    if(x==LOW)//checks the light intensity and turn OFF the led
        {
          digitalWrite(led,LOW);
        }
    else
        {
          Serial.print("Now correct light intensity obtained that is :");
          Serial.println(x);
        }
    if(t>30)//checks the temperature and turn ON the fan
    {
      digitalWrite(fan1, HIGH);
      digitalWrite(fan2, HIGH);
      digitalWrite(fan3, HIGH);
    }
    if(t<29)//checks the temperature and turn OFF the fan
    {
       digitalWrite(fan1,LOW);
       digitalWrite(fan2,LOW);
       digitalWrite(fan3,LOW);
        }

  digitalWrite(LED_PIN, HIGH);  // Turn LED ON
}
 if (signal == '0')//human is not found
```

```
    {
      Serial.println(F("No human detected. Skipping sensor readings."));
      digitalWrite(fan1, LOW);
      digitalWrite(fan2, LOW);
      digitalWrite(fan3, LOW);// Ensure fan is OFF
      digitalWrite(led, LOW);  // Ensure LED is OFF
      digitalWrite(LED_PIN, LOW);   // Turn LED OFF
    }
  }
}
```

2. Python Code

- **Save the following code as Human_Detection.py inside the YOLOv5-master directory:**

```
import cv2
import torch
import serial
import time
import sys  # For termination

# Initialize YOLOv5 model
model = torch.hub.load('ultralytics/yolov5', 'yolov5s')  # Load YOLOv5s model
model.classes = [0]  # Filter for "person" class (COCO ID 0)qq

# Initialize serial communication
arduino = serial.Serial(port='COM4', baudrate=9600, timeout=1)  # Replace 'COM3' with your Arduino's port
time.sleep(2)  # Wait for Arduino to initialize

# Initialize webcam
cap = cv2.VideoCapture(0)

# Key for termination
TERMINATION_KEY = 'q'
print(f"Press '{TERMINATION_KEY}' to terminate the process.")

try:
    while True:
        ret, frame = cap.read()
```

```
        if not ret:
            print("Error: Unable to read from the webcam.")
            break

        # Perform object detection
        results = model(frame)

        # Check if a person is detected
        detected = any(results.xyxy[0][:, -1] == 0)  # Check if class '0' (person) is in the detections

        # Send signal to Arduino
        signal = '1' if detected else '0'
        arduino.write(signal.encode())

        # Display results
        cv2.imshow("YOLOv5 Detection", results.render()[0])

        # Break on termination key
        if cv2.waitKey(1) & 0xFF == ord(TERMINATION_KEY):
            print("Termination key pressed. Exiting...")
            break

except Exception as e:
    print(f"An error occurred: {e}")

finally:
    # Release resources
    print("Releasing resources...")
    cap.release()
```

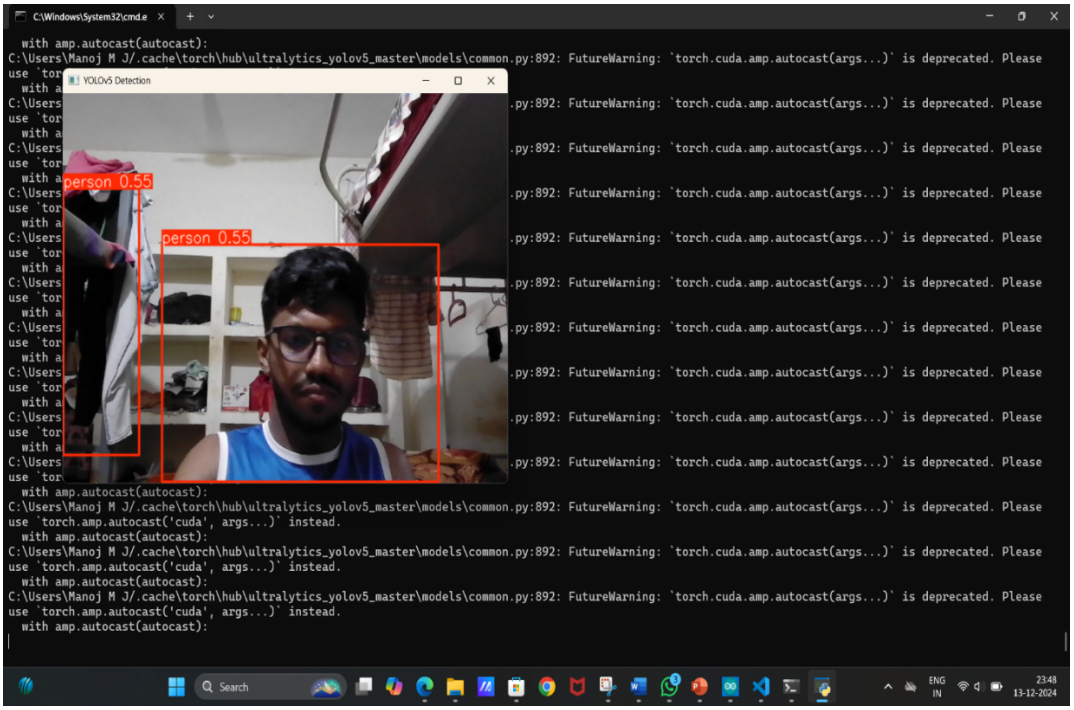
```
cv2.destroyAllWindows()
arduino.write('0'.encode())  # Ensure LED is turned OFF
    arduino.close()
    print("Process terminated.")
```

Running the System

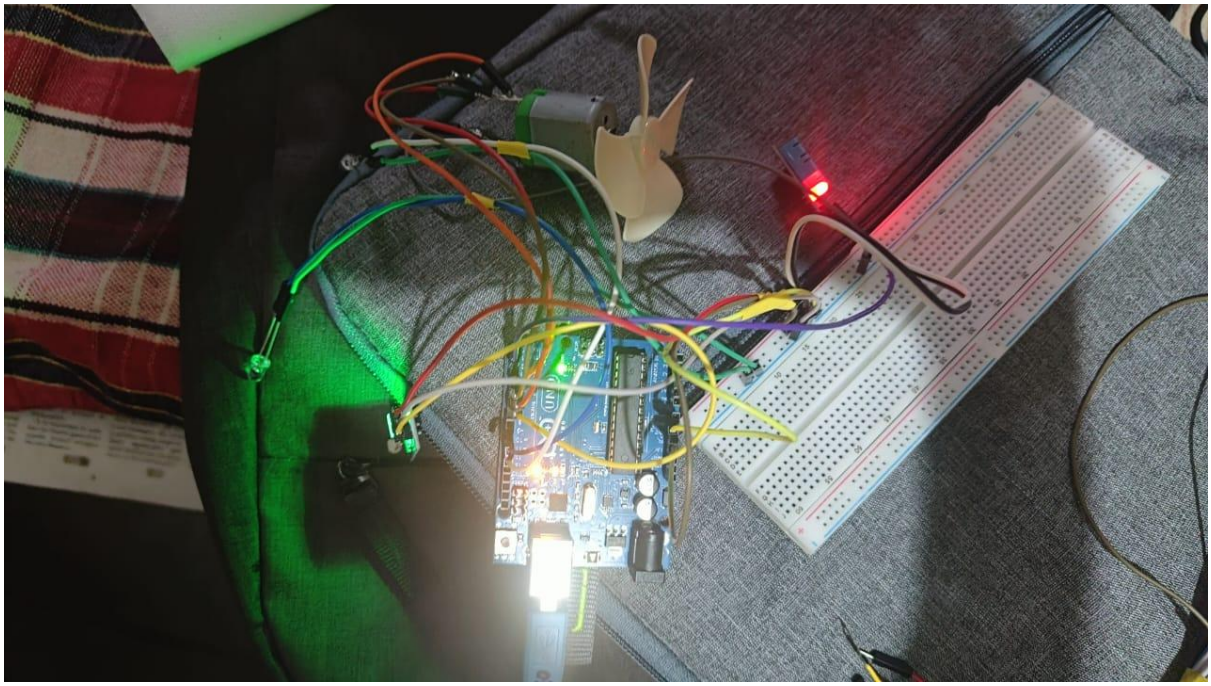
1. **Upload the Arduino code using the Arduino IDE.**
2. **Navigate to the YOLOv5-master directory in the terminal.**
3. **Run the Python script in terminal:**

```
python Demo_LED_Test6.py
```

Human Detection:



A typical Arduino and with their peripherals:



Temperature Monitoring:
Temperature sensors (e.g., DHT11 or DHT22) continuously monitor the ambient temperature. The system uses this data to regulate HVAC systems intelligently.

- For example, if the temperature exceeds a predefined threshold, cooling systems are activated or adjusted to ensure comfort while minimizing energy consumption.

- **Light Intensity Monitoring**:
  Light-dependent resistors (LDRs) or similar sensors measure ambient light levels. The system adjusts artificial lighting based on natural light availability.
  - **If sufficient natural light is available**, artificial lights are dimmed or turned off.
  - **If the ambient light level drops**, artificial lights are automatically adjusted to provide adequate illumination.

- **Machine Learning Integration**:
  Machine learning algorithms analyse data from sensors over time to recognize patterns in human presence, temperature fluctuations, and light intensity.
  - **Optimization**: It adjusts settings dynamically to balance energy efficiency and user comfort.
  - **Adaptation**: The system improves its decision-making accuracy by learning from real-world usage scenarios.
- **Automation and Control**:
  The entire process is automated, ensuring that the system responds in real-time without the need for manual intervention. IoT-enabled devices facilitate remote monitoring and control, allowing users to oversee and adjust settings through smartphones or computers.
- **Energy Saving and Feedback**:
  The system continuously evaluates energy usage, providing detailed feedback to users. This promotes awareness and encourages sustainable energy habits.

This smart integration of human detection, environmental monitoring, and machine learning makes the system an efficient, intelligent, and environmentally friendly solution for modern energy management.

5. Advantages:

Smart Electricity Management, driven by human detection, temperature monitoring, and light intensity measurement with machine learning, offers several advantages:

- **Enhanced Energy Efficiency**:

- Automatically turns off electrical devices in unoccupied spaces.
- Adjusts heating, cooling, and lighting based on environmental conditions, ensuring minimal energy wastage.

- **Cost Savings**:
  - Reduces electricity bills by optimizing the operation of devices based on real-time requirements and predictive analytics.
- **Personalized Comfort**:
  - Maintains optimal temperature and lighting conditions tailored to user preferences and environmental factors, enhancing comfort.
- **Environmental Benefits**:
  - Reduces carbon footprint by conserving energy and promoting sustainable energy practices.
  - Encourages the efficient use of renewable energy sources by integrating them into the system.
- **Real-time Monitoring and Control**:
  - Provides users with real-time insights into energy consumption through IoT-enabled dashboards or apps.
  - Allows remote monitoring and control of devices, offering convenience and flexibility.

By combining cutting-edge technology with practical applications, **Smart Electricity Management** not only optimizes energy usage but also supports a sustainable and user-friendly approach to modern living.

6. Challenges:

Implementing **Smart Electricity Management** systems based on human detection, temperature monitoring, and light intensity with machine learning techniques comes with several challenges:

- **Accuracy of Human Detection**:
  - Detecting human presence reliably, especially in low-light or obstructed environments, can be challenging.
  - False positives or negatives in detection may lead to inefficient operation of devices.
- **Sensor Limitations**:
  - Environmental sensors such as DHT11, LDRs, and cameras may face issues like wear and tear, calibration errors, or sensitivity to external disturbances (e.g., dust, heat, or vibrations).
  - Limited range or low-quality sensors can reduce the overall system performance.

Addressing these challenges requires careful design, robust technology, user education, and supportive regulatory frameworks to maximize the effectiveness of **Smart Electricity Management** systems.

7. Applications:

Smart Electricity Management, utilizing human detection, temperature monitoring, and light intensity measurement powered by machine learning, finds extensive applications across various domains. Here are the key applications:

- **Smart Homes**

- **Automated Lighting**: Adjust lighting levels based on occupancy and ambient light.
- **HVAC Control**: Optimize heating and cooling systems based on room temperature and presence of individuals.
- **Energy Monitoring**: Provide real-time insights into energy usage to promote efficient habits.

➢ Commercial Buildings **and Offices**

- **Energy-efficient Workspaces**: Automate lights, HVAC systems, and projectors in meeting rooms based on occupancy.
- **Cost Reduction**: Minimize utility bills by reducing waste in unused office spaces.

➢ **Industrial Settings**

- **Smart Manufacturing**: Control machinery and ventilation systems based on environmental conditions and human presence.

➢ **Educational Institutions**

- **Smart Classrooms**: Control projectors, lights, and air conditioning based on class schedules and occupancy.
- **Energy Conservation Programs**: Use real-time data for teaching energy efficiency practices to students.

➢ **Healthcare Facilities**

- **Optimized Patient Rooms**: Adjust lighting and temperature to maintain comfort based on patient presence and needs.
- **Energy-efficient Operations**: Manage HVAC and lighting systems in waiting areas, operation theaters, and unused rooms.

These applications demonstrate the versatility of **Smart Electricity Management** systems, showcasing their ability to optimize energy use, enhance user convenience, and promote sustainability in a wide range of environments.

8. Future Scope:

The future of **Smart Electricity Management**, driven by human detection, temperature monitoring, light intensity measurement, and machine learning techniques, is highly promising. Advancements in technology and increasing emphasis on energy efficiency and sustainability will expand its applications and effectiveness. Key future directions include:

- **Integration with Renewable Energy Sources**
  - Seamless integration of solar panels, wind turbines, and other renewable energy sources into the smart grid.
  - Intelligent decision-making for optimal energy distribution between renewable and conventional sources.
- **Advanced Machine Learning Models**
  - Utilization of deep learning and reinforcement learning for more accurate predictions of energy demand and user behaviour.
  - Self-learning algorithms that adapt to new environments and optimize energy usage dynamically.
- **IoT and Smart Grid Enhancements**
  - Broader adoption of IoT-enabled devices to enhance real-time monitoring and control.
  - Customization of systems for various climates, building types, and cultural preferences.

➢ **Government Policies and Incentives**

- Support from governments and organizations through incentives for adopting smart energy solutions.
- Establishment of global standards and guidelines for smart electricity management systems.

➢ **Integration with Energy Storage**

- Smart management of battery systems to store excess energy during low-demand periods for future use.

The future scope of **Smart Electricity Management** lies in its potential to revolutionize energy usage across homes, industries, and cities. With continuous advancements in machine learning and IoT, the system is poised to become a cornerstone of sustainable and intelligent energy practices.

9. Conclusion:

"Smart Electricity Management", driven by human detection, temperature monitoring, light intensity measurement, and machine learning techniques, represents a significant step forward in energy optimization and sustainability. By intelligently monitoring and controlling energy usage based on real-time environmental and occupancy data, this system ensures maximum efficiency, reduced energy wastage, and enhanced user comfort.

The integration of machine learning enables the system to learn from usage patterns and improve over time, making it highly adaptive and reliable. With applications ranging from homes and offices to industries and public infrastructure, it has the potential to revolutionize the way energy is managed in diverse settings.

Despite challenges such as sensor limitations, privacy concerns, and integration complexities, advancements in technology and increasing

awareness about energy conservation provide a robust foundation for overcoming these obstacles. The adoption of **Smart Electricity Management** not only reduces operational costs but also contributes significantly to global sustainability goals by lowering the carbon footprint.

In conclusion, **Smart Electricity Management** is a forward-looking solution that aligns with the growing need for energy-efficient, intelligent, and eco-friendly systems. It holds the promise of transforming energy consumption practices and shaping a more sustainable future.